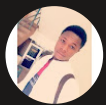


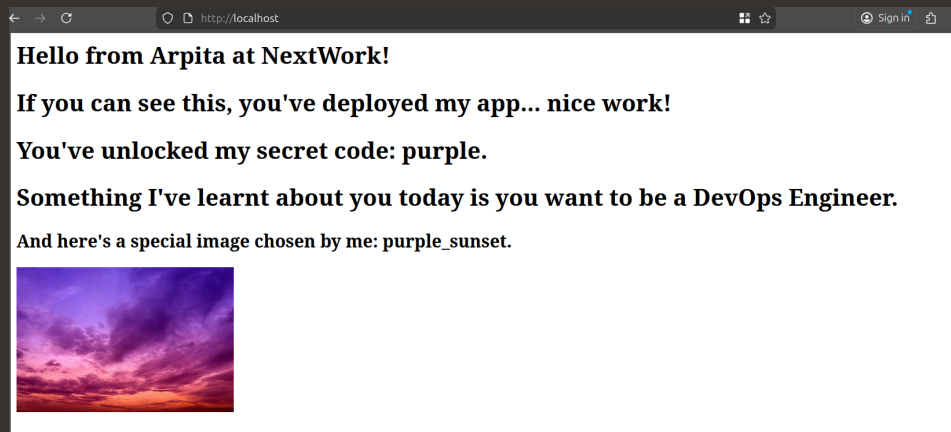


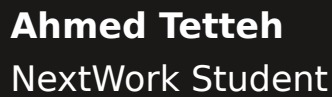
nextwork.org

Deploy an App Across Accounts



Ahmed Tetteh



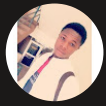


What is Amazon ECR?

One thing I didn't expect...

This project took me...

This project took me approximately 4 hours. My biggest learning was the docker image system architecture type, Dockerrun.aws.json file ,i.e, instructions for



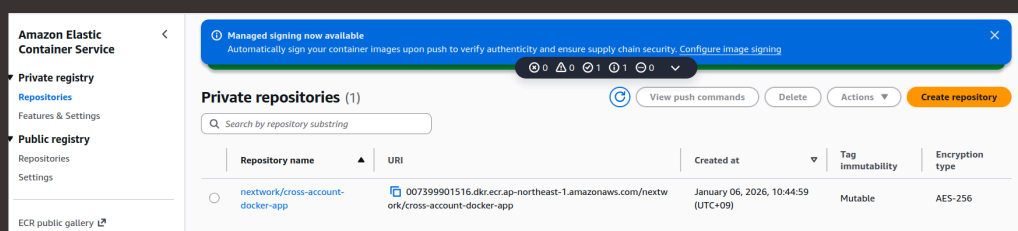
Creating a Docker Image

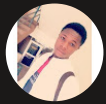
I set up both the dockerfile and my custom index.html file(my application code). Both files are needed because they will be used to create the docker that my project buddy will deploy on their end.

My Dockerfile tells Docker to fetch the base image Nginx from the container registry and replace the default html file that comes with it, with my custom index.html file.

I also set up an ECR repository

ECR stands for Elastic Container Registry. It is important because it allows developers to store and manage the deployment of container images, making images easier to track and secure.





Set Up AWS CLI Access

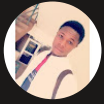
AWS CLI can let me run ECR commands

AWS CLI is tool that lets you manage your AWS services from your terminal. The CLI asked for my credentials because it needs to connect to my AWS account.

To enable CLI access, I set up a new IAM user with the permission `AmazonEC2ContainerRegistryFullAccess`. I also set up an access key for this user, which means I will be able to connect to my AWS account through this IAM with the access key.

To pass my credentials to the AWS CLI, I ran the command "aws configure" to put in my access key details for the IAM user I created.

```
AWS Access Key ID [None]:  
AWS Secret Access Key [None]:  
Default region name [None]:  
Default output format [None]:
```



Pushing My Image to ECR

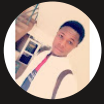
Push commands are commands that push an a docker image into a repository.

There are three main push commands

To authenticate Docker with my ECR repo, I used the command "aws ecr get-login-password --region ap-northeast-1 | docker login --username AWS --password-stdin 007399901516.dkr.ecr.ap-northeast-1.amazonaws.com" to get the credentials for ECR and pass those credentials to Docker for access to the repository.

To push my container image, I ran the command "docker push 007399901516.dkr.ecr.ap-northeast-1.amazonaws.com/nextwork/cross-account-docker-app:latest". Pushing means uploading our docker image to a container repository.

When I built my image, I tagged it with the label. latest.. This means any container I run will be using the image with the tag- latest

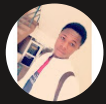


Resolving Permission Issues

When I tried pulling my project buddy's container image for the first time, I saw the error 403 Forbidden response. This was because the ECR repositories are private, hence, specific permissions need to be added to pull images from that repository.

To resolve each other, we added our IAM user's ARN to the ECR repository policy.

```
king@king-TFG5577XG:~/Desktop/Personal Lab /DockerECR$ docker pull 006834263464.dkr.ecr.us-east-1.amazonaws.com/nextwork/cross-account-docker-app:latest
Error response from daemon: unknown: failed to resolve reference "006834263464.dkr.ecr.us-east-1.amazonaws.com/nextwork/cross-account-docker-app:latest": unexpected status from HEAD request to https://006834263464.dkr.ecr.us-east-1.amazonaws.com/v2/nextwork/cross-account-docker-app/manifests/latest: 403 Forbidden
```

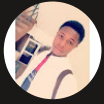


Deploying the App

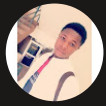
I used Elastic Beanstalk to deploy the web app. When I set up an Elastic Beanstalk application, I configured settings like Compute capacity for my application, managed updates, health checks and more.

While setting up for deployment, I created a new role for my EC2 instance and Elastic Beanstalk... The role has the permission for Elastic Beanstalk to perform healthchecks and more, while my EC2 instance. I gave Elastic Beanstalk access to ECR to fetch the docker image from the ECR repository.

The Dockerrun.aws.json file is a file that tells Elastic Beanstalk how to deploy a container image from repositories like ECR. My file tells Elastic Beanstalk to deploy a single container of my application, the image name and expose the container for access on port 80.



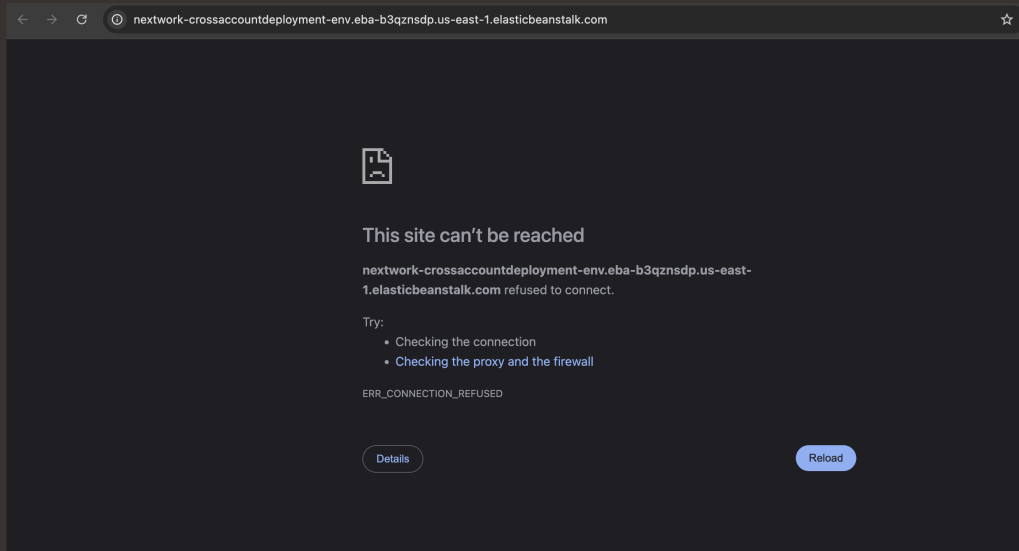
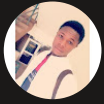
```
king@king-TFG5577XG:~/Desktop/Personal Lab /DockerECR$ cat Dockerrun.aws.json
{
  "AWSEBDockerrunVersion": "1",
  "Image": {
    "Name": "006834263464.dkr.ecr.us-east-1.amazonaws.com/nextwork/cross-account-docker-app:latest",
    "Update": "true"
  },
  "Ports": [
    {
      "ContainerPort": "80"
    }
  ]
}
```



Resolving Deployment Issues

When I visited my environment and clicked on the domain URL, I run into a page error saying "This site can't be reached". This was expected because, previously in Amazon ECR, we only set permissions for a specific IAM user to access the ECR repository, not Elastic Beanstalk and the EC2 instance.

To fix the permissions error, my buddy and I modified the permissions for the ECR repository by adding the ARNs of both Elastic Beanstalk and the EC2 instance. So, once this is done the Elastic Beanstalk environment is rebuilt with the new updates, we should be able to access eachother's sites.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

